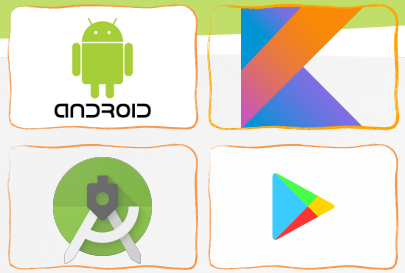
Software Development for Mobile Devices

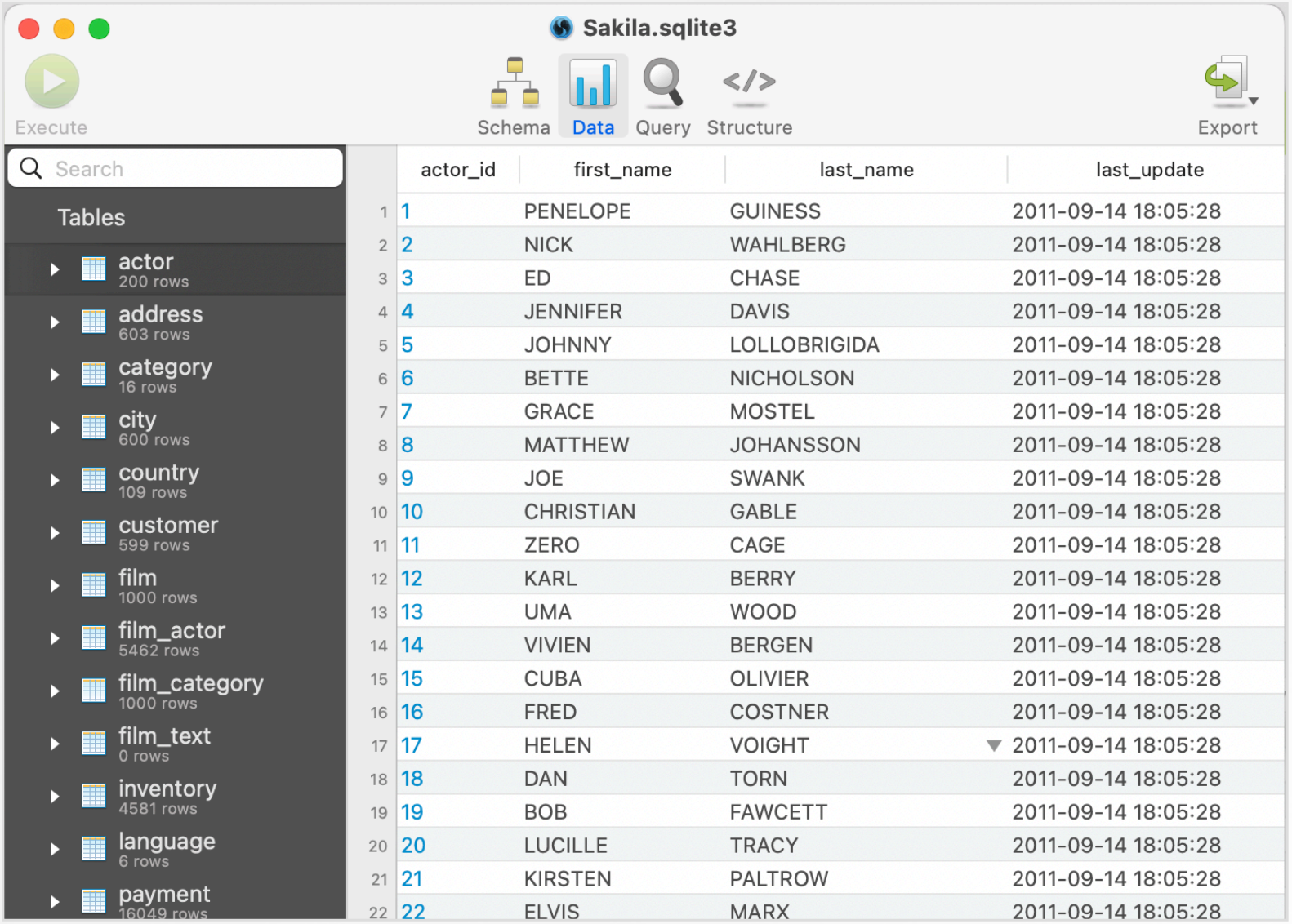
Nguyen Le Hoang Dung
nlhdung@fit.hcmus.edu.vn

SQLite



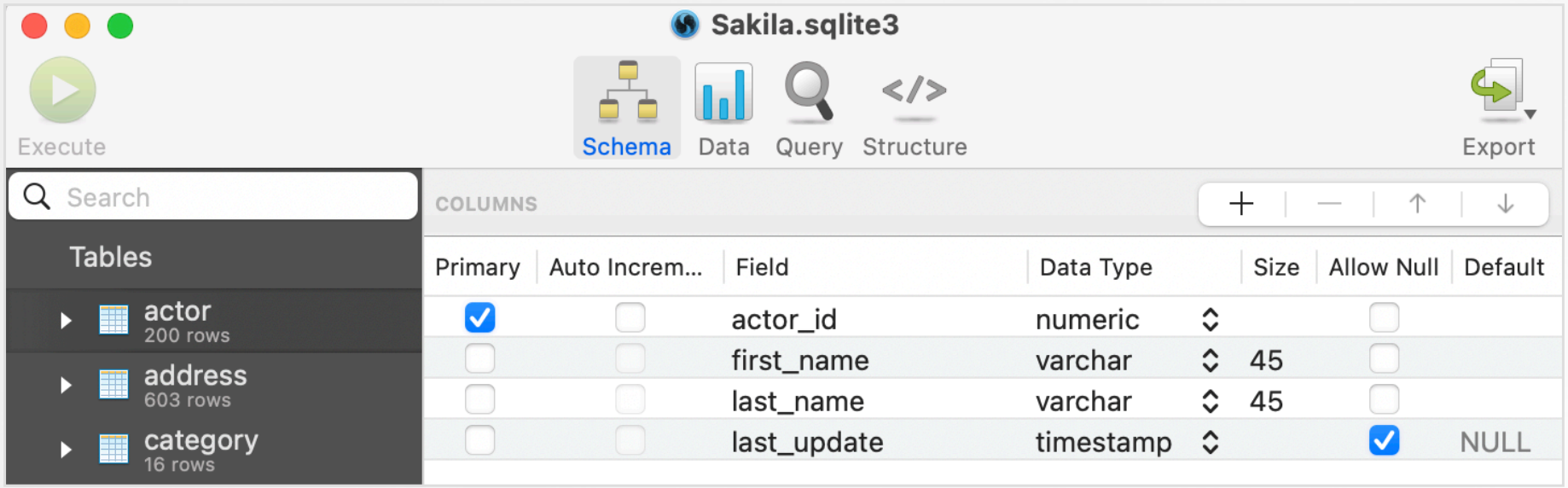
- ***SQLite*** is an embedded, relational database management system (RDBMS) that stores data to a text file on a device.
- The SQLite file format is stable, cross-platform which is built into all mobile phones and most computers.
- Android comes in with built in SQLite database implementation (***android.database.sqlite*** package)
- SQLite supports all the relational database features.
  - In order to access this database, we don't need to establish any kind of connections for it like JDBC (Java Database Connectivity), ODBC (Open Database Connectivity)…

SQLite

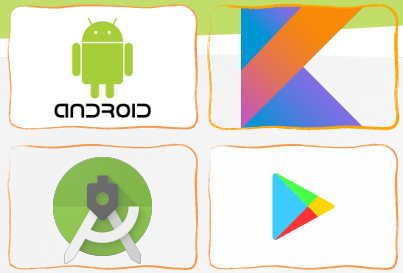

Sakila.sqlite3
Execute
Schema
Data
Query
Structure
Export
Search
Tables
actor 200 rows
address 603 rows
category 16 rows
city 600 rows
country 109 rows
customer 599 rows
film 1000 rows
film_actor 5462 rows
film_category 1000 rows
film_text 0 rows
inventory 4581 rows
language 6 rows
payment 16049 rows
actor_id | first_name | last_name | last_update
1 | 1 | PENELOPE | GUINESS | 2011-09-14 18:05:28
2 | 2 | NICK | WAHLBERG | 2011-09-14 18:05:28
3 | 3 | ED | CHASE | 2011-09-14 18:05:28
4 | 4 | JENNIFER | DAVIS | 2011-09-14 18:05:28
5 | 5 | JOHNNY | LOLLOBRIGIDA | 2011-09-14 18:05:28
6 | 6 | BETTE | NICHOLSON | 2011-09-14 18:05:28
7 | 7 | GRACE | MOSTEL | 2011-09-14 18:05:28
8 | 8 | MATTHEW | JOHANSSON | 2011-09-14 18:05:28
9 | 9 | JOE | SWANK | 2011-09-14 18:05:28
10 | 10 | CHRISTIAN | GABLE | 2011-09-14 18:05:28
11 | 11 | ZERO | CAGE | 2011-09-14 18:05:28
12 | 12 | KARL | BERRY | 2011-09-14 18:05:28
13 | 13 | UMA | WOOD | 2011-09-14 18:05:28
14 | 14 | VIVIEN | BERGEN | 2011-09-14 18:05:28
15 | 15 | CUBA | OLIVIER | 2011-09-14 18:05:28
16 | 16 | FRED | COSTNER | 2011-09-14 18:05:28
17 | 17 | HELEN | VOIGHT | 2011-09-14 18:05:28
18 | 18 | DAN | TORN | 2011-09-14 18:05:28
19 | 19 | BOB | FAWCETT | 2011-09-14 18:05:28
20 | 20 | LUCILLE | TRACY | 2011-09-14 18:05:28
21 | 21 | KIRSTEN | PALTROW | 2011-09-14 18:05:28
22 | 22 | ELVIS | MARX | 2011-09-14 18:05:28

SQLite

» The table schema: a description of the table that is contained within the database



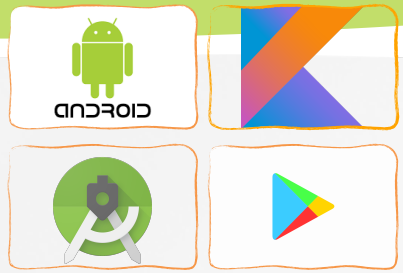
Room persistence library for SQLite



» The Room persistence library provides an abstraction layer over SQLite to allow fluent database access while harnessing the full power of SQLite:

- **Compile-time verification of SQL queries**.
  - Each @Query and @Entity is checked at the compile time, that preserves your app from crash issues at runtime and not only it checks the only syntax, but also missing tables.
- **Minimize repetitive and boilerplate code**.
  - Room takes care of a lot of the repetitive code such as writing SQL queries and parsing cursors. This allows you to focus on the logic of your app instead of dealing with low-level database details.

Room persistence library for SQLite



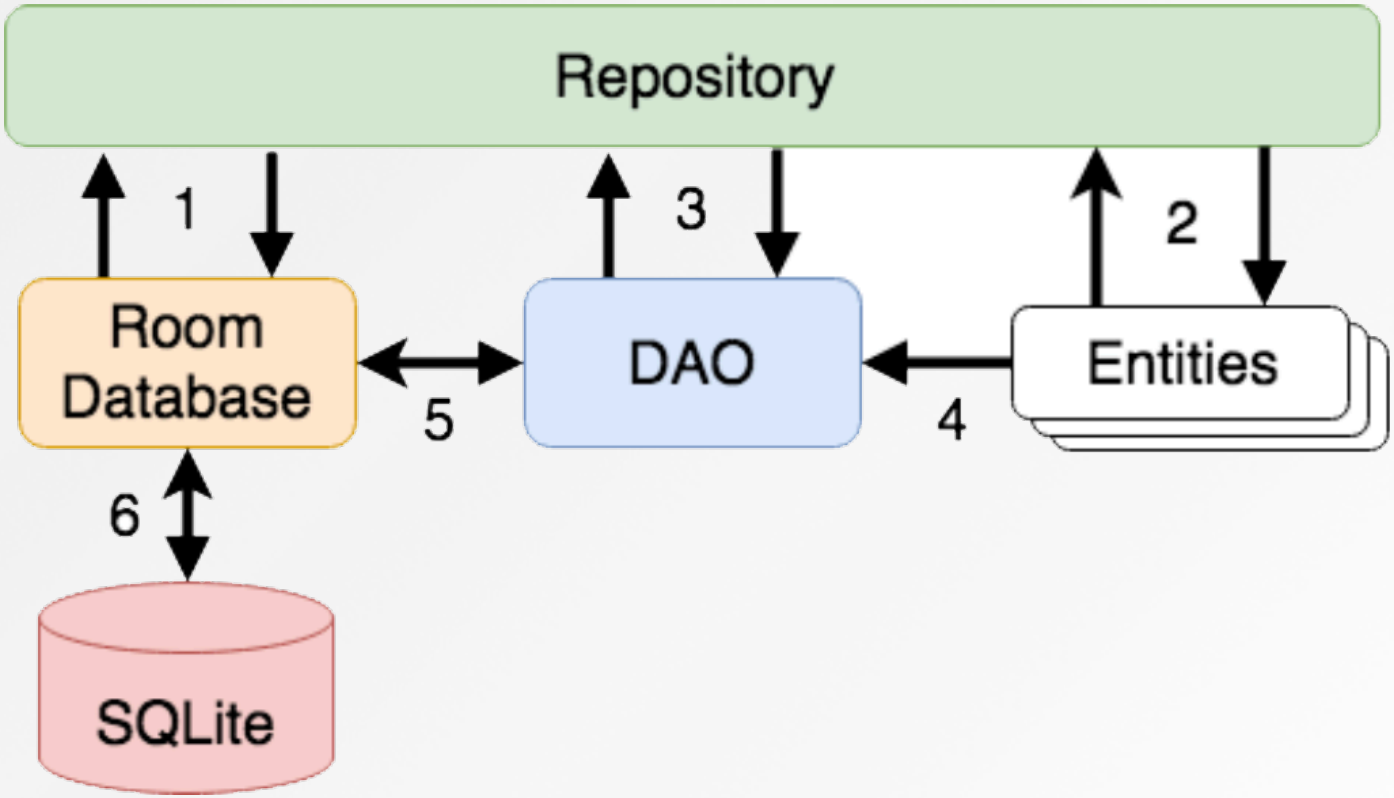
» The Room persistence library provides an abstraction layer over SQLite to allow fluent database access while harnessing the full power of SQLite:

- **Object Mapping:** Room allows you to map database tables to Java/Kotlin objects using annotations. As a result, you can access it using familiar object-oriented principles
- **LiveData Integration:** Room is built to work with LiveData and RxJava allowing you to observe changes in the database and update your UI automatically, while SQLite does not.

Components of Room DB

» The SQLite DB responsible for storing & providing access to the data.

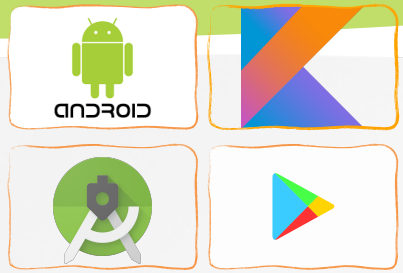
» The app code (including Repository) should never make direct access to the database. All database operations are performed using a combination of the room database, DAOs and entities.



Components of Room DB
Entity

```
@Entity(tableName = "customers")
class Customer {
    @PrimaryKey(autoGenerate = true)
    @NonNull
    @ColumnInfo(name = "customerId")
    var id: Int = 0

    @ColumnInfo(name = "customerName")
    var name: String? = null

    var address: String? = null

    constructor() {}
        constructor(id: Int, name: String, address: String) {
        this.id = id; this.name = name; this.address = address
    }
    constructor(name: String, address: String) {
        this.name = name; this.address = address
    }
}
```

Components of Room DB
Entity

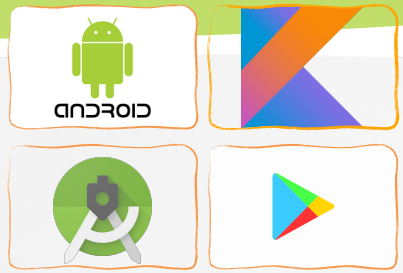


» It represents a table within the database.
  – Room creates a table for each class that has `@Entity` annotation, the fields in the class correspond to columns in the table.

» Annotations
  – **@Entity** — every model class with this annotation will have a mapping table in DB
    • tableName: ***@Entity(tableName = "customers")***
    • **@PrimaryKey** — points the primary key of the entity.
      • autoGenerate — if set to true, then SQLite will be generating a unique id for the column

    @PrimaryKey(autoGenerate = true)

Components of Room DB
Entity

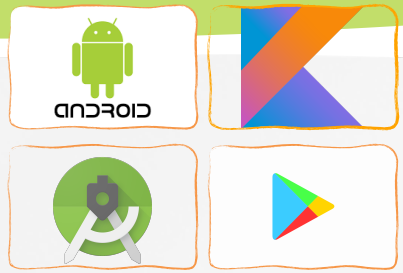


» Annotations

- **@NonNull:** cannot be used to store null values
- **@ColumnInfo — specifying custom information about column**

  @ColumnInfo(name = "column_name")
- **@Ignore** — field will not be persisted by Room (not required to be stored)

Components of Room DB
Entity

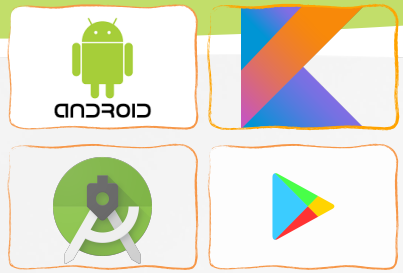


» Annotations

– **@Embeded** — nested fields can be referenced directly in the SQL queries.

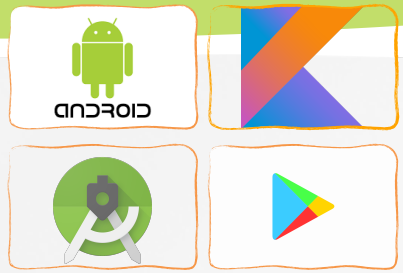
- **Nested Objects:** You can define a class (e.g., `Address`) with its own fields (`street`, `city`, `zip`…).
- **Embed in Entity:** In your Room entity `User`, you include an instance of that class as a field and annotate it with `@Embedded`.
- **Flattened Columns:** Room will automatically create columns in the database table for the entity (`User`) corresponding to the fields of the embedded class (`Address`). You can access these fields directly in your queries.

Components of Room DB
Entity



```
data class Address(
    val street: String,
    val city: String,
    val zip: Int
)

@Entity
data class User(
    @PrimaryKey
    val id: Int,

    val name: String,

    @Embedded
    val address: Address
)

@Dao
interface UserDao {
    @Query("SELECT * FROM User WHERE address.city = :city")
    fun getUsersByCity(city: String): List<User>
}
```

Components of Room DB
Entity

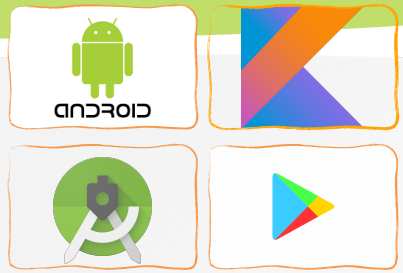


» Annotations

– foreignKeys — names of foreign keys

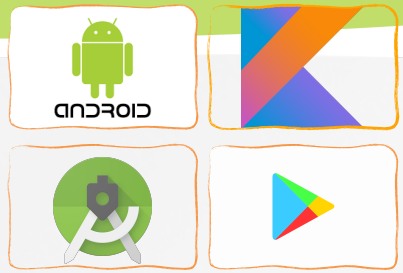
```
@Entity(foreignKeys = arrayOf(ForeignKey(entity = Customer::class,
parentColumns = arrayOf("customerId"),
childColumns = arrayOf("buyerId"),
onDelete = ForeignKey.CASCADE,
onUpdate = ForeignKey.RESTRICT)))
class Purchase {
    @PrimaryKey(autoGenerate = true)
    @NonNull
    @ColumnInfo(name = "purchaseId")
    var purchaseId: Int = 0
    @ColumnInfo(name = "buyerId")
    var buyerId: Int = 0
    .
    .
}
```

Components of Room DB
DAO



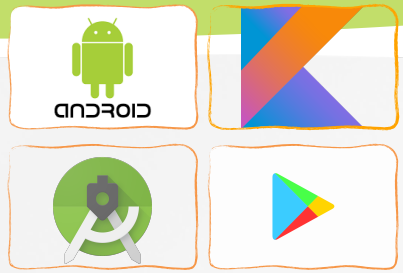
- They are responsible for defining the methods that access the database.
- A DAO (Data Access Object) provides a way to access the data stored within a SQLite database.
- A DAO is declared as a standard Kotlin interface with some additional annotations that map specific SQL statements to methods that may then be called by the repository.

Components of Room DB Database



- The Room database class is created by extending the RoomDatabase class and acts as a layer on top of the actual SQLite database embedded into the Android operating system. The class is responsible for creating and returning a new room database instance and for providing access to the DAO instances associated with the database.
- To create a database we need to:
  - Define an abstract class that extends ***`RoomDatabase`***. It is annotated with ***`@Database`***
  - Thi class lists the entities contained in the database, and the DAOs which access them.

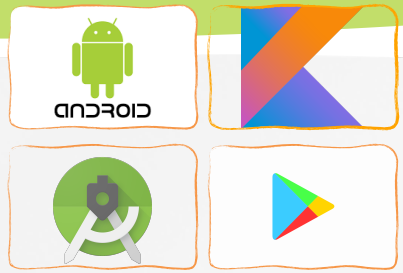
Components of Room DB Database



» The class that's annotated with `@Database` should satisfy the following conditions:

- Be an abstract class that extends `RoomDatabase`.
- Include the list of entities associated with the database within the annotation.
- Contain an abstract method that has 0 arguments and returns the class that is annotated with `@Dao`.

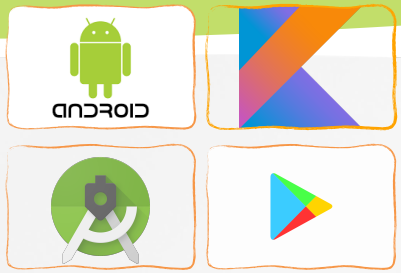
Components of Room DB Database



» At runtime, you can acquire an instance of Database by calling:

- ***Room.databaseBuilder()***: this database exists as a database file on the persistent storage of an Android device
- ***Room.inMemoryDatabaseBuilder()***: these databases reside entirely in memory and are lost when the app terminate

Example 1
Implementation of Room

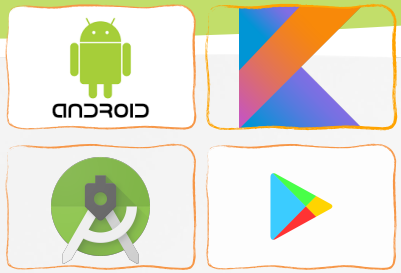


» Add the Gradle dependencies

– Open your ***app's build.gradle*** file file and add these lines

```
plugins {
    id 'com.android.application'
    id 'org.jetbrains.kotlin.android'
    id 'kotlin-kapt'
}
//..
dependencies {
    implementation 'androidx.core:core-ktx:1.7.0'
    implementation 'androidx.appcompat:appcompat:1.4.1'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.3'


    implementation "androidx.room:room-runtime:2.4.2"
    kapt 'androidx.room:room-compiler:2.4.2'
}
```

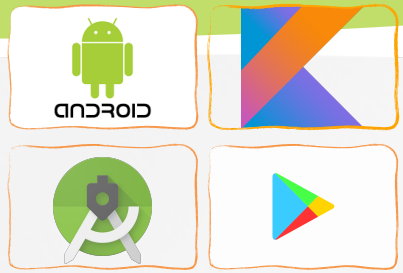
Example 1
Implementation of Room



» Create model class

```
import androidx.room.ColumnInfo
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "person")
data class Person (
    @ColumnInfo(name = "name" )
    var name : String,
    @ColumnInfo(name = "city")
    var city : String,
){
    @PrimaryKey(autoGenerate = true)
    var id: Int = 0
}
```

Example 1
Implementation of Room

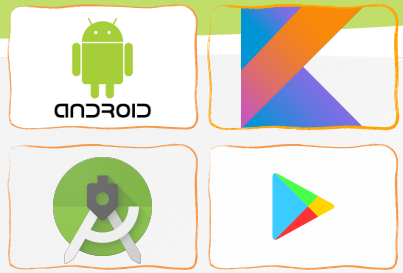


» Create DAO

```
import androidx.room.*

@Dao
public interface PersonDAO {
    @Query("Select * from person")
    fun getPersonList() : List<Person>

    @Query("SELECT * FROM person WHERE name LIKE :name")
    fun findByName(name: String): Person

    @Insert
    fun insertPerson(person : Person)

    @Update
    fun updatePerson(person: Person)

    @Delete
    fun deletePerson(person: Person)
}
```

https://developer.android.com/reference/kotlin/androidx/room/Dao

Example 1
Implementation of Room

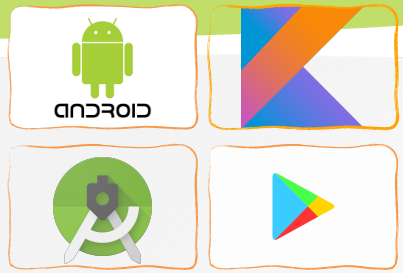


» Create the database

```
import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = arrayOf(Person::class), version = 1)
abstract class PersonDatabase : RoomDatabase() {
    abstract fun personDao() : PersonDAO

    companion object {
        val DB_NAME = "person_db"
        private var instance: PersonDatabase? = null
        fun getInstance(context: Context): PersonDatabase {
            return instance ?: buildDatabase(context).also { instance = it}
        }

         private fun buildDatabase(context: Context) =
             Room.databaseBuilder(context, PersonDatabase::class.java,
             DB_NAME).allowMainThreadQueries().build()
    }
}
```

https://developer.android.com/reference/kotlin/androidx/room/Database

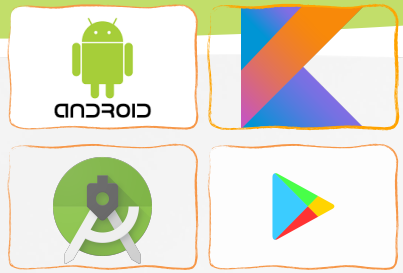
Example 1
Implementation of Room



» Let's test our classes

```
fun testRoomDB(){
    //Create DB
    db = PersonDatabase.getInstance(this)
    Log.i("hdlog", getDatabasePath(PersonDatabase.DB_NAME).absolutePath)

    //Insert
    testInsertPerson()
    testPrintAllPerson()

    testQueryAndDeletePerson()
    testPrintAllPerson()

    //Update persion1
    val person1 = Person("a1", "HCM city")
    person1.id = 1
    db.personDao().updatePerson(person1)

    testPrintAllPerson()
    db.close()    //Close db
}
```

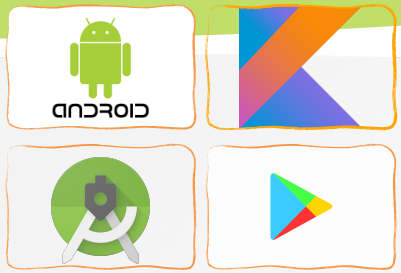
Example 1
Implementation of Room



» Let's test our classes

```
fun testInsertPerson(){
    //Insert
    val person1 = Person("a1","HCM")
    val person2 = Person("a2","Thu Duc")
    val person3 = Person("a3","Ben Tre")
    db.personDao().insertPerson(person1)
    db.personDao().insertPerson(person2)
    db.personDao().insertPerson(person3)
}

fun testPrintAllPerson(){
    val list = db.personDao().getPersonList()
    Log.i("hdlog", "-------------")
    for (i in 0..list.size-1) {
        Log.i("hdlog", list[i].toString())
    }
}
```

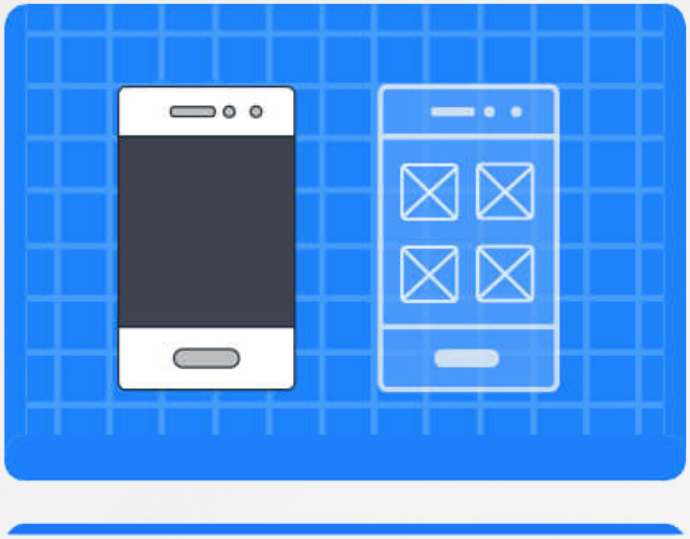
Example 1
Implementation of Room



» Let's test our classes

```
fun testQueryAndDeletePerson(){
    //Query
    val person2FromDB = db.personDao().findByName("a2")
    if (person2FromDB != null) {
        Log.i("hdlog", "Found: " + person2FromDB.toString())
        db.personDao().deletePerson(person2FromDB)
    }

    val person3FromDB = db.personDao().findByName("a3333")
    if (person3FromDB != null)
        Log.i("hdlog", "Found: " + person3FromDB.toString())
    else{
        Log.i("hdlog", "Cannot find Person3 from DB")
    }
}
```

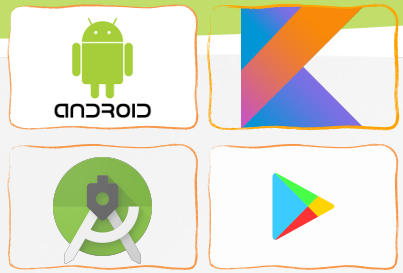
Demo



Kotlin

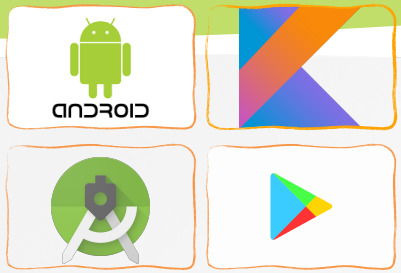
Q&A

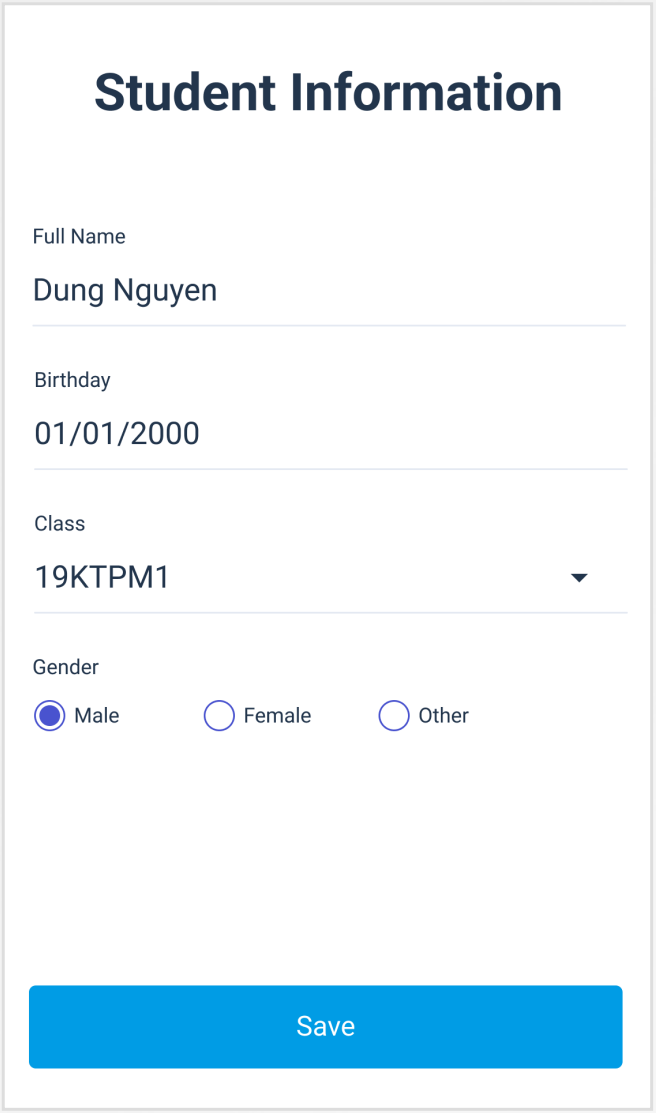
Homework
Student Management App (version 3)



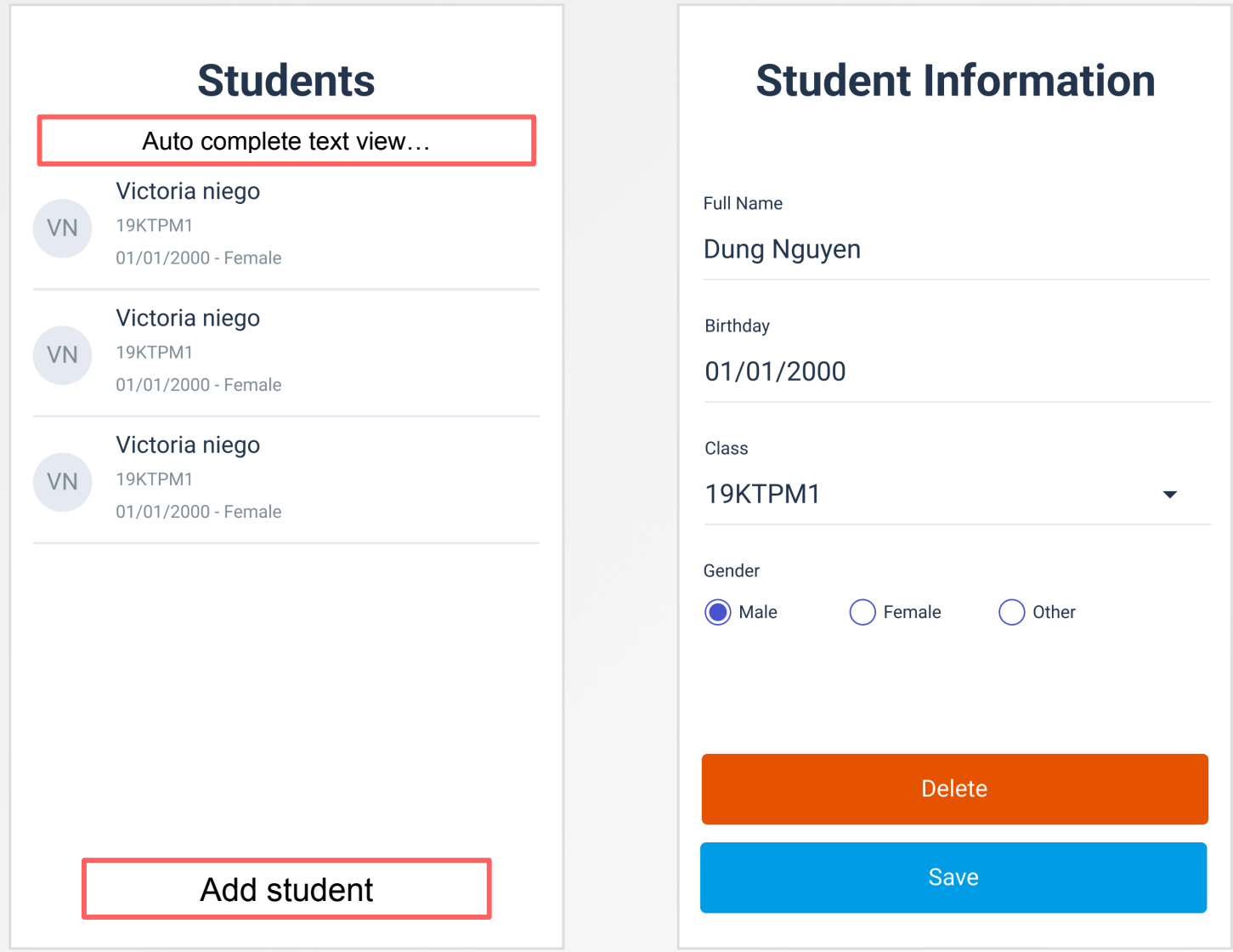
- Input student's information (full name; birthday, class, gender)
  - Save student's information using text file (with your own format)
- Display list of students
- Select a student and
  - edit student's information
  - delete that student and reload list of students
- **Version 2:**
  - **Use RecyclerView for displaying list of students**
  - **Use Spinner for selecting a Class**
  - **Add a EditText using for searching student with AutoComplete feature (student's name) (in List of students Activity)**
  - **Touch a button for changing layout: grid layout or linear layout**
  - **Use Sqlite or Realm**

Homework
Student Management App (version 3)

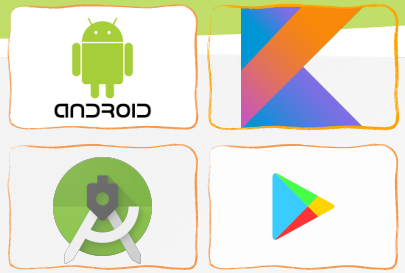




Homework
Student Management App (version 3)



Reference



- https://kotlinlang.org/docs/reference/
- Peter Späth, Learn Kotlin for Android Development, 2019
- Ted Hagos, Learn Android Studio 3 with Kotlin, 2018
- Peter Späth, Pro Android with Kotlin, 2019
- Milos Vasic, Mastering Android Development with Kotlin, 2017
- https://www.tutorialspoint.com/android/android_sqlite_database.htm
- https://developer.android.com/reference/android/arch/persistence/room/package-summary
- https://developer.android.com/training/data-storage/room